HERMES · PHASE 0 · LESSON 0004 · LAB EXERCISE 03-STREAMING-AND-CANCELLATION · ~45–60 MIN

The Response Becomes a Process

Exercise 02's request gains one field, `stream: true`. The response stops being a value you await. It becomes a process you watch, and today you stop one mid-way.

Why this matters for Hermes

Scenario step S6 (budget enforcement) aborts a model call while the answer is still being written. Text never generated is never billed, so this abort is the budget's enforcement. Today you build and measure it against the mock. The lesson advances lesson 0001's thread of six responsibilities: responsibility ⑥ (cancellation) changes meaning.

1 · One response, many events

The request changes by one body field. The answer changes shape. Instead of one JSON document, the server answers `200` with `content-type: text/event-stream` and no `content-length`, because the total length is not known yet.

Server-sent events are a plain-text format for pushing many events over one held-open HTTP response. Each event is an `event:` line naming a type and a `data:` line of JSON, separated by blank lines (MDN):

```
event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":"Hello from the mock."}}
```

The event types and their order are fixed (streaming reference). The mock sends the same sequence, readable in both terminals.

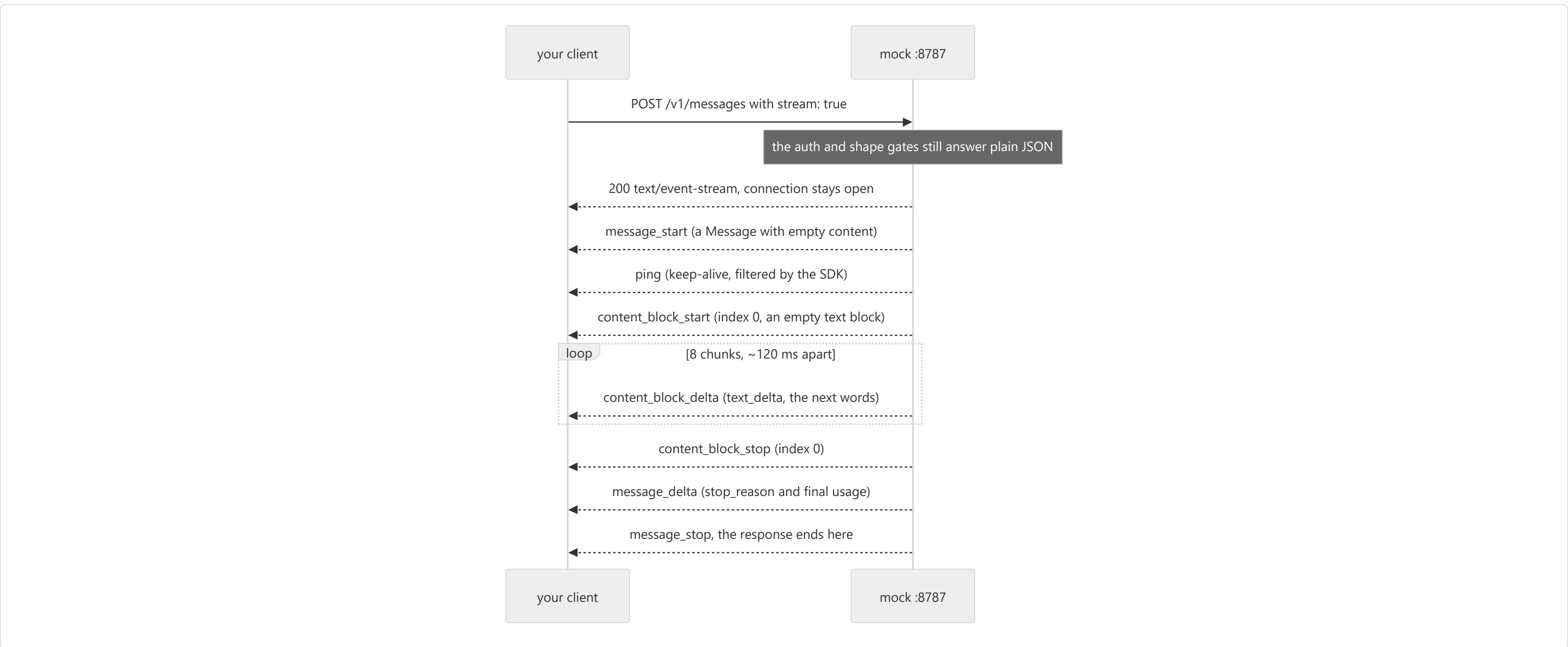
your client
mock :8787
POST /v1/messages with stream: true
the auth and shape gates still answer plain JSON
200 text/event-stream, connection stays open
message_start (a Message with empty content)
ping (keep-alive, filtered by the SDK)
content_block_start (index 0, an empty text block)
loop
[8 chunks, ~120 ms apart]
content_block_delta (text_delta, the next words)
content_block_stop (index 0)
message_delta (stop_reason and final usage)
message_stop, the response ends here
your client
mock :8787

Diagram 1. The streaming events, as the mock sends them. Measured: 13 events reach the client over ~1.4 s. The SDK filters the `ping` out before your code iterates.

New today.

Five ideas are new.

- A response can arrive as many typed events over one held-open connection.
- Each event carries one slice of the `Message`, in a fixed order.
- The prompt's cost arrives first. The ending and the output cost arrive last.
- The SDK offers raw event iteration and an assembling helper.
- An abort mid-stream keeps the partial and stops the remainder.

Reprise: the mock and its gates (lesson 0002), narrowing on a type field (lesson 0001), the abort signal (lesson 0003).

2 · Each event carries one slice

The six event types are not interchangeable packets. Each carries one slice of the `Message` you know from exercises 01 to 03.

Event	What it carries	Why there
`message_start`	a `Message` with `id`, `model` and `usage.input_tokens`, but `content: []` and `stop_reason: null`	what the server knows before generating
`content_block_start`	an empty block and its `index`	opens a slot for deltas to patch
`content_block_delta`	a `text_delta`, the next few words	the generation itself, as it happens
`content_block_stop`	the `index` alone	closes the slot
`message_delta`	`stop_reason` and the final `usage.output_tokens`	what the server knows only after generating
`message_stop`	nothing	the process is over, and the connection closes

A *delta* is an incremental patch against a message that is still being assembled. Assemble the slices and nothing is missing. Start with `message_start`'s value, append every delta, then add `message_delta`'s two fields. The result is the `Message` the non-streaming path returns in one piece.

The lab measures it: the assembled text is byte-for-byte exercise 02's fixture, with the same `usage` and `stop_reason`. Streaming changes delivery. The response shape does not change.

The order is information. The count `usage.input_tokens` rides the first event, because the server priced the prompt before answering. The `stop_reason` and the real `output_tokens` ride the last events. While a stream is in flight, nobody knows what it will cost or how it will end.

The six responsibilities from lesson 0001 barely move today. The table says which layers changed.

Layer	Today
① endpoint	unchanged. The same `POST /v1/messages`
② auth and ③ version	unchanged. The gates answer before any stream
④ request and response shape	one request field added. A successful response becomes `text/event-stream`
⑤ errors and retries	unchanged. Failures answer as plain JSON, before a stream begins
⑥ cancellation and timeout	transformed. §4 measures it

3 · Two SDK surfaces, one wire format

The SDK reads this wire at two levels. Both are declared in the installed `.d.ts` files, the authoritative description.

```
// resources/messages/messages.d.ts. The params type picks the overload.
create(params: MessageCreateParamsNonStreaming, … ): APIPromise<Message>;
create(params: MessageCreateParamsStreaming, … ):  APIPromise<Stream<RawMessageStreamEvent>>;

// lib/MessageStream.d.ts. The helper.
declare class MessageStream implements AsyncIterable<MessageStreamEvent> {
  on(event: "text", listener: (textDelta: string, textSnapshot: string) ⇒ void): this;
  finalMessage(): Promise<Message>;   // assembles §2's slices for you
  finalText(): Promise<string>;
  get currentMessage(): Message | undefined;  // the partly assembled Message, mid-flight
  get request_id(): string | null | undefined;
  abort(): void;
}
```

One word now names four things. Keep them apart.

The phrase	Level	Meaning
`stream: true`	request field	asks for event delivery on the wire
`Stream<RawMessageStreamEvent>`	return type	the sequence of raw events, one by one
`client.messages.stream()`	method	the helper's entry point. Its params carry no `stream: true`
`MessageStream`	class	the helper object that assembles the events

An *async iterator* is an object that yields values one at a time, each one awaited. With `stream: true`, `create()` returns one. Every yielded event is a union member, so `for await` plus narrowing on `event.type` is lesson 0001's `block.type` move, one level up. Assembly is your job.

A *message stream* is the SDK helper that assembles streamed events into a `Message` for you. The call `client.messages.stream(params)` returns one. The `.on("text")` listener receives each chunk and the running total. The `finalMessage()` method resolves to the assembled `Message`.

The result of `finalMessage()` carries `_request_id: undefined`. Nothing is broken. That object never traveled as one HTTP body, because the helper assembled it client-side from events. The request id belongs to the HTTP exchange, so it lives on the stream object as `stream.request_id`.

4 · The abort lands mid-process

In exercises 01 to 03, an abort interrupted a wait. Nothing had arrived, so nothing was kept. Today the abort lands in the middle of generation. The lab fires it at 700 ms.

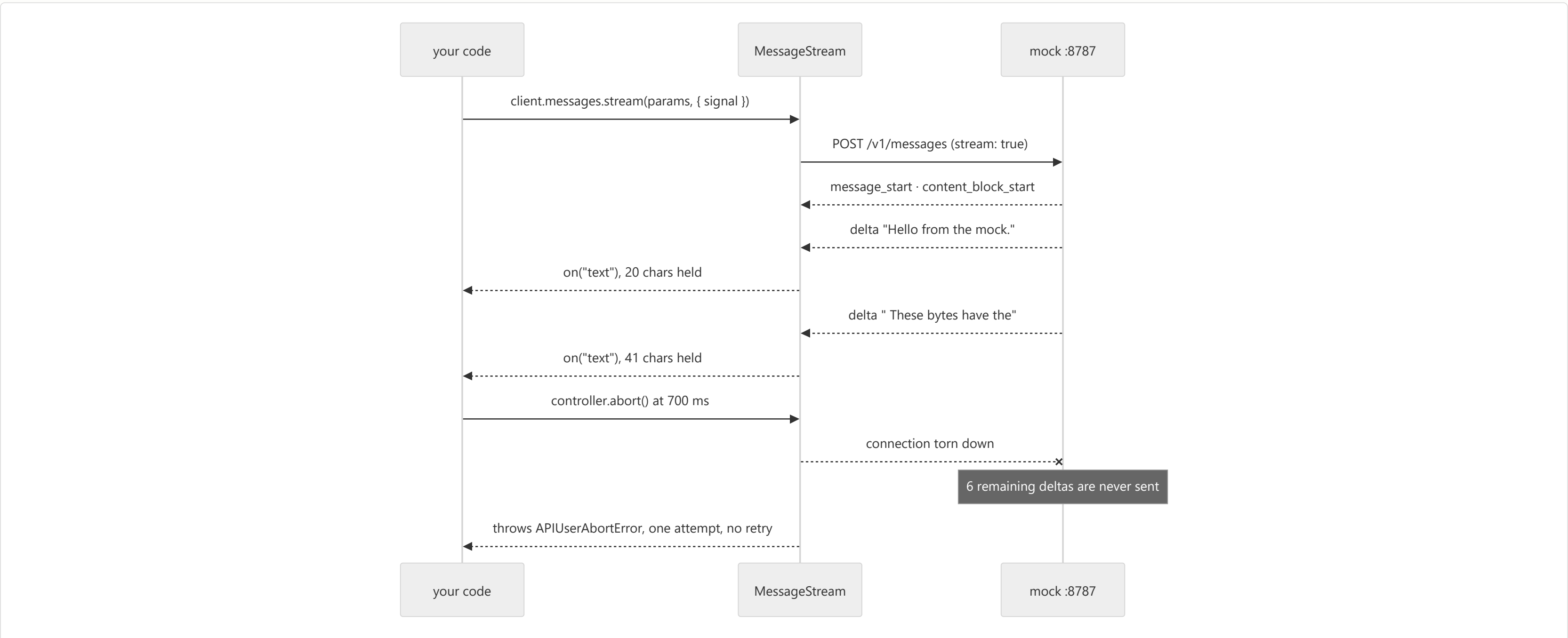
your code
MessageStream
mock :8787
client.messages.stream(params, { signal })
POST /v1/messages (stream: true)
message_start · content_block_start
delta "Hello from the mock."
on("text"), 20 chars held
delta " These bytes have the"
on("text"), 41 chars held
controller.abort() at 700 ms
connection torn down
6 remaining deltas are never sent
throws APIUserAbortError, one attempt, no retry
your code
MessageStream
mock :8787

Diagram 2. Measured: `APIUserAbortError` ~30 ms after the abort. The client holds 41 of 164 characters. The mock logs two of eight deltas, then the disconnect.

You keep the partial. The text that arrived is yours. There is no `stop_reason`, because the message never reached its end. The property `stream.currentMessage` still shows the partly assembled value: two chunks, `stop_reason: null`.

The remainder is never generated. Against the real API, ungenerated text is unbilled text. Step S6 uses this abort as its enforcement: at the cost ceiling, Hermes stops the answer while the model is writing it.

The SDK does not retry a user abort. The mock logs one attempt, unlike lesson 0003's timeouts.

Capability	Status	Where it lives, or when it arrives
A streamed response, event by event	implemented	the mock's stream branch, Part A
The helper's assembly and `finalMessage()`	implemented	Part B
An abort that keeps the partial text	implemented	Part C, fired at 700 ms
Budget enforcement on this abort, step S6	implemented	lesson 0009's lab, `07-tool-loop`
Tool-use events inside a stream	implemented	the mock, since lesson 0009. Not exercised here
An alert at 80% of a ceiling	planned	Phase 1, per `ROADMAP.md`

5 · The exercise

Commands and the full experiment table live in `hermes-sdk-lab/03-streaming-and-cancellation/README.md`. The arc:

1. Read the declarations first. Open `resources/messages/messages.d.ts` in the installed SDK and find the two `create` overloads. Then skim `lib/MessageStream.d.ts`. Every §3 claim is on those pages.
2. Part A: add `stream: true`, iterate with `for await`, and log every `event.type`. Write the order down before checking Diagram 1. Find which event carries `stop_reason`.
3. Part B: call `client.messages.stream()`. Print the chunks from `.on("text")`, then `finalMessage()` and exercise 02's four printouts. One printout differs. Find the id's home.
4. Part C: abort at 700 ms. Catch `APIUserAbortError` and count the characters you kept. Watch the mock's terminal.
5. Close by saying which of the six responsibilities moved. Five barely move. Responsibility ⑥ (cancellation) transforms.

6 · Exercise: when does the stream know it?

Place each fact in the event that delivers it

Answer from memory before you click. Feedback is immediate.

`usage.input_tokens`, the cost of reading your prompt.

message_start, before generation | content_block_delta, during generation | message_delta, after generation

`stop_reason`, how the answer ended.

message_start, before generation | content_block_delta, during generation | message_delta, after generation

The words of the answer.

message_start, before generation | content_block_delta, during generation | message_delta, after generation

The `msg_…` id and the model that is answering.

message_start, before generation | content_block_delta, during generation | message_delta, after generation

The real `usage.output_tokens`, the cost of the answer.

message_start, before generation | content_block_delta, during generation | message_delta, after generation

The moment a 700 ms abort interrupts.

message_start, before generation | content_block_delta, during generation | message_delta, after generation

Score: 0 / 0

7 · Quiz

Four questions

Q1. You add `stream: true` to exercise 02's params. What changed?

The request gates, since extra checks now guard the endpoint

The delivery of the response, while the content is unchanged

The model's output, which now writes shorter text pieces

The client construction, since streaming needs its own client

Q2. Mid-stream, with deltas still arriving, which of these does the client already know for certain?

The stop_reason, since the server decided it before starting

The output_tokens, since usage arrived in the first event

The input_tokens, since the prompt was counted up front

The final length, since content_length was in the headers

Q3. `finalMessage()` returns a message whose `_request_id` is `undefined`. Why?

The object was assembled client-side, and the id stays on the stream

The mock omits request-id headers on all of its streaming responses

The SDK strips identifiers from every message once a stream closes

The id only exists on errors, and success paths never carry one

Q4. You abort 700 ms into the stream. What is the outcome, per the lab's measurements?

Nothing is kept, because the SDK discards partial text on aborts

All text arrives anyway, because the server finishes what it began

The SDK retries twice, because aborts obey the usual retry policy

The partial stays yours, and the remainder is never generated

Score: 0 / 0

8 · Say it in your own words

Answer these in the chat after the lab, with no notes.

1. Define server-sent events in one sentence. Say what stays open that normally closes.
2. Walk the event order from memory: which event opens, what patches what, and which two facts arrive only at the end. Say why they cannot arrive earlier.
3. Compare raw `for await` iteration with the helper. Say what the helper does for you, and name one Hermes use for each surface.
4. Exercise 02's abort produced nothing. Today's kept 41 characters and left the rest ungenerated. Say what changed about responsibility ⑥, and why a budget without a mid-stream abort is only advisory.

If you keep only four sentences, keep these. With `stream: true` the same request is answered as typed events over one held-open connection. Assemble the events and you hold the `Message` the plain path returns in one piece. The prompt's cost arrives in the first event, the ending and the output cost in the last. An abort keeps the text that arrived, and the ungenerated remainder is never billed, which makes a budget enforceable.

Primary source

Read the Messages API streaming reference and check each event shape against your terminal's log. Then read `node_modules/@anthropic-ai/sdk/lib/MessageStream.d.ts` top to bottom, about 120 lines.

Version pin. Every excerpt and number here was verified against `@anthropic-ai/sdk` 0.113.0, by live runs against the exercise 01 mock. The mock's pacing is synthetic: eight deltas about 120 ms apart, where the real API paces by generation speed. It does not simulate `thinking_delta`. The four new terms have notes under `wiki/terms/`.

Stuck or curious? Ask your teacher why the SDK filters pings, or how `snapshot` differs from `delta` in `.on("text")`.

Next: lesson 0005. Every response type so far is an assertion over parsed bytes. Zod turns that claim into a checked parse, and closes Phase 0.